MIPT & YSDA · AUTUMN 2026

# Deep Generative Models

## Lecture 10

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

# Recap of Previous Lecture

## Continuous-Time Dynamics

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \text{with initial condition } \mathbf{x}(t_0) = \mathbf{x}_0.$$

$$\boldsymbol{\psi}_t(\mathbf{x}_0) = \int_{t_0}^{t} \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(s), s) ds + \mathbf{x}_0 \approx \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0, t).$$

- $\mathbf{v}_{\boldsymbol{\theta}} : \mathbb{R}^m \times [t_0, t_1] \to \mathbb{R}^m$ is a vector field.
- $\boldsymbol{\psi} : \mathbb{R}^m \times [t_0, t_1] \to \mathbb{R}^m$ is a flow (the solution of ODE):

## Euler `ODESolve`

$$\mathbf{x}(t + h) = \mathbf{x}(t) + h \cdot \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

For a smooth field on a fixed interval, compare global error and **NFE** (velocity-network evaluations):

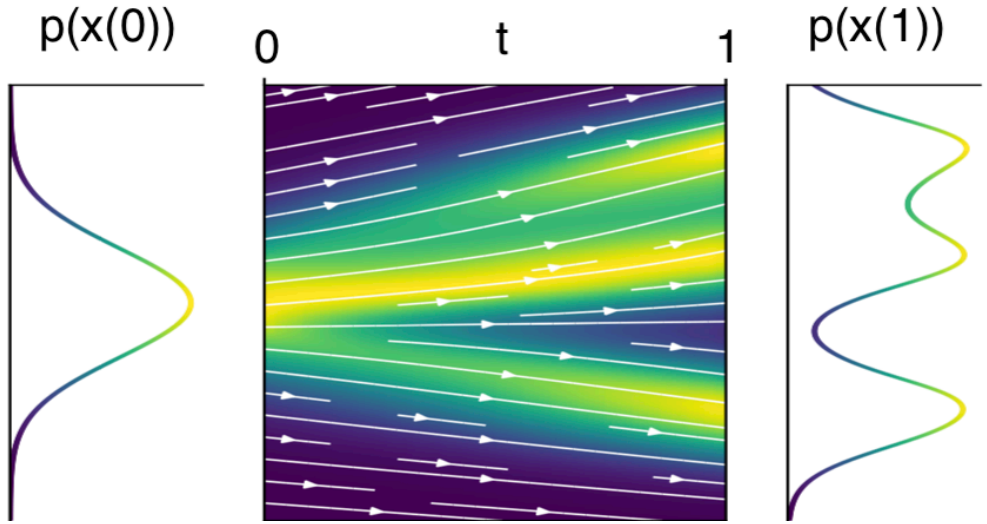
- **Euler:** $\mathcal{O}(h)$ global error, **1** evaluation per step.
- **Heun:** $\mathcal{O}(h^2)$ global error, **2** evaluations per step, averaging the current and predicted endpoint velocities.

Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Driscoll T. A., Braun R. J. Fundamentals of Numerical Computation: Runge–Kutta methods

# Recap of Previous Lecture

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \mathbf{x}(t_0) = \mathbf{x}_0$$

- Suppose $\mathbf{x}(0)$ is a random variable with density $p_0(\mathbf{x})$. Then, $\mathbf{x}(t)$ is a random variable with density $p_t(\mathbf{x})$.
- $p_t(\mathbf{x}) = p(\mathbf{x}, t)$ describes the **probability path** between $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$.



# Recap of Previous Lecture

### Theorem (Picard)

If $\mathbf{v}$ is continuously differentiable with a bounded derivative in $\mathbf{x}$ and continuous in $t$, then the ODE has a **unique solution** given by a flow $\boldsymbol{\psi}_t$.

This guarantees the ODE is **uniquely reversible**.

$$\boldsymbol{\psi}_1(\mathbf{x}_0) = \mathbf{x}_0 + \int_0^1 \mathbf{v}_{\boldsymbol{\theta}}(\boldsymbol{\psi}_t(\mathbf{x}_0), t)dt$$

$$\mathbf{x}(1) = \mathbf{x}(0) + \int_0^1 \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)dt$$

$$\mathbf{x}(0) = \mathbf{x}(1) + \int_1^0 \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)dt$$

**Note:** Unlike discrete-time flows, $\mathbf{v}$ need not be invertible (uniqueness ensures bijection).

How can we compute $p_t(\mathbf{x})$ at arbitrary $t$?

# Recap of Previous Lecture

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \text{with initial condition } \mathbf{x}(t_0) = \mathbf{x}_0$$

## Theorem (Continuity Equation)

If $\mathbf{v}$ is continuously differentiable in $\mathbf{x}$ (with bounded derivative) and continuous in $t$, and $p_t(\mathbf{x}) > 0$, then

$$\frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right)$$

## Solution

$$\log p_1(\mathbf{x}(1)) = \log p_0(\mathbf{x}(0)) - \int_0^1 \operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt.$$

## Hutchinson's trace estimator:

$$\operatorname{tr}(\mathbf{J}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})}[\boldsymbol{\epsilon}^T \mathbf{J} \boldsymbol{\epsilon}], \qquad \mathbf{J} = \frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}.$$

**Exact density identity; approximate computation.** Use a numerical ODE solver and one noise vector for an unbiased trace estimate via autodiff.

Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

# Recap of Previous Lecture

Fix $p_0 = \mathcal{N}(0, \mathbf{I})$ and learn $\mathbf{v}_{\boldsymbol{\theta}}$; the endpoint density is $p_1 = p_{\boldsymbol{\theta}}$.

## Training

1. Sample $\mathbf{x} \sim p_{\text{data}}$. Solve the ODE **backward**, from $\mathbf{x}(1) = \mathbf{x}$ to $\mathbf{x}(0)$.
2. Compute the negative log-likelihood along this trajectory:

$$\mathcal{L} = -\log p_{\boldsymbol{\theta}}(\mathbf{x}) = -\log p_0(\mathbf{x}(0)) + \int_0^1 \operatorname{tr}\left(\frac{\partial \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt.$$

3. Update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$; differentiate through the ODE, e.g., via the **adjoint method**.

## Sampling

1. Sample $\mathbf{x}(0) \sim p_0$.
2. Solve the ODE **forward** and return the endpoint:

$$\mathbf{x}(1) = \texttt{ODESolve}_v(\mathbf{x}(0), \boldsymbol{\theta}, t_0 = 0, t_1 = 1) \sim p_{\boldsymbol{\theta}}.$$

Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

# Recap of Previous Lecture

## SDE Basics

Let's define a stochastic process $\mathbf{x}(t)$ with initial condition $\mathbf{x}(0) \sim p_0(\mathbf{x})$:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w},$$

where $\mathbf{w}(t)$ is the standard Wiener process (Brownian motion):

$$\mathbf{w}(t) - \mathbf{w}(s) \sim \mathcal{N}(0, (t-s)\mathbf{I}), \quad d\mathbf{w} = \boldsymbol{\epsilon} \cdot \sqrt{dt}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

## Discretizing the SDE (Euler-Maruyama Update) – `SDESolve`

$$\mathbf{x}(t + dt) = \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t) \cdot dt + g(t) \cdot \boldsymbol{\epsilon} \cdot \sqrt{dt}$$

- At each time $t$, we have the density $p_t(\mathbf{x}) = p(\mathbf{x}, t)$.
- $p : \mathbb{R}^m \times [0, 1] \to \mathbb{R}_+$ is a **probability path** between $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$.

# Recap of Previous Lecture

### Theorem (Kolmogorov-Fokker-Planck)

If $p_t(\mathbf{x}) \in C^{1,2}$ (i.e., $C^1$ in $t$ and $C^2$ in $\mathbf{x}$), then

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right) + \frac{1}{2}g^2(t)\Delta_{\mathbf{x}} p_t(\mathbf{x})$$

### Langevin SDE (Special Case)

$$d\mathbf{x} = \frac{1}{2}\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})dt + 1 \cdot d\mathbf{w}$$

The density $p_t(\mathbf{x})$ is a **stationary** distribution for the SDE. Setting the stationary density to the model distribution, $p_t(\mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x})$ for all $t$:

### Langevin Dynamic

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \frac{\eta}{2} \cdot \nabla_{\mathbf{x}} \log p_{\boldsymbol{\theta}}(\mathbf{x}) + \sqrt{\eta} \cdot \boldsymbol{\epsilon}, \quad \eta \approx dt.$$

# Outline

01 Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)

02 Probability Flow ODE

03 Reverse SDE

04 Score-Based Generative Models Through SDEs

05 Denoiser Parameterization and Sampling

06 Flow Matching (FM)

# Outline

01 **Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)**

02 Probability Flow ODE

03 Reverse SDE

04 Score-Based Generative Models Through SDEs

05 Denoiser Parameterization and Sampling

06 Flow Matching (FM)

# Score Matching SDE

## Denoising Score Matching

$$\mathbf{x}_t = \mathbf{x} + \sigma_t \cdot \boldsymbol{\epsilon}_t, \qquad q(\mathbf{x}_t|\mathbf{x}) = \mathcal{N}(\mathbf{x}, \sigma_t^2 \cdot \mathbf{I})$$
$$\mathbf{x}_{t-1} = \mathbf{x} + \sigma_{t-1} \cdot \boldsymbol{\epsilon}_{t-1}, \quad q(\mathbf{x}_{t-1}|\mathbf{x}) = \mathcal{N}(\mathbf{x}, \sigma_{t-1}^2 \cdot \mathbf{I})$$

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \cdot \boldsymbol{\epsilon}, \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_{t-1}, (\sigma_t^2 - \sigma_{t-1}^2) \cdot \mathbf{I})$$

Let's transform this Markov chain into the continuous stochastic process $\mathbf{x}(t)$ by letting $T \to \infty$:

$$\begin{aligned}
\mathbf{x}(t) &= \mathbf{x}(t - dt) + \sqrt{\sigma^2(t) - \sigma^2(t - dt)} \cdot \boldsymbol{\epsilon} \\
&= \mathbf{x}(t - dt) + \sqrt{\frac{\sigma^2(t) - \sigma^2(t - dt)}{dt} dt} \cdot \boldsymbol{\epsilon} \\
&= \mathbf{x}(t - dt) + \sqrt{\frac{d[\sigma^2(t)]}{dt}} \cdot d\mathbf{w}
\end{aligned}$$

# Score Matching SDE

$$\mathbf{x}(t) = \mathbf{x}(t - dt) + \sqrt{\frac{d[\sigma^2(t)]}{dt}} \cdot d\mathbf{w}$$

$$d\mathbf{x} = \mathbf{x}(t) - \mathbf{x}(t - dt)$$

## Variance Exploding SDE

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} \cdot d\mathbf{w}$$

$\sigma(t)$ is a monotonically increasing function.

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

$$\mathbf{f}(\mathbf{x}, t) = 0, \quad g(t) = \sqrt{\frac{d[\sigma^2(t)]}{dt}}$$

# Diffusion SDE

## Denoising Diffusion

$$\mathbf{x}_t = \sqrt{1-\beta_t} \cdot \mathbf{x}_{t-1} + \sqrt{\beta_t} \cdot \boldsymbol{\epsilon}_t, \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t} \cdot \mathbf{x}_{t-1}, \beta_t \cdot \mathbf{I})$$

Let's turn this Markov chain into a continuous stochastic process by letting $T \to \infty$ and setting $\beta_t = \beta(\frac{t}{T}) \cdot \frac{1}{T}$ (where $dt = \frac{1}{T}$):

$$\begin{aligned}
\mathbf{x}(t) &= \sqrt{1-\beta(t)dt} \cdot \mathbf{x}(t-dt) + \sqrt{\beta(t)dt} \cdot \boldsymbol{\epsilon} \\
&\approx \left(1 - \frac{1}{2}\beta(t)dt\right) \cdot \mathbf{x}(t-dt) + \sqrt{\beta(t)dt} \cdot \boldsymbol{\epsilon} \\
&= \mathbf{x}(t-dt) - \frac{1}{2}\beta(t)\mathbf{x}(t-dt)dt + \sqrt{\beta(t)} \cdot d\mathbf{w}
\end{aligned}$$

# Diffusion SDE

$$\mathbf{x}(t) = \mathbf{x}(t - dt) - \frac{1}{2}\beta(t)\mathbf{x}(t - dt)dt + \sqrt{\beta(t)} \cdot d\mathbf{w}$$

## Variance Preserving SDE

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)} \cdot d\mathbf{w}$$

$$\mathbf{f}(\mathbf{x}, t) = -\frac{1}{2}\beta(t)\mathbf{x}(t), \quad g(t) = \sqrt{\beta(t)}$$

Variance is preserved as long as $\mathbf{x}(0)$ has unit variance.

Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

# Diffusion SDE

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

## Variance Exploding SDE (NCSN)

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} \cdot d\mathbf{w}$$

## Variance Preserving SDE (DDPM)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)} \cdot d\mathbf{w}$$

We treat the discrete-time methods (NCSN, DDPM) as a special case of the continuous-time methods (VE-SDE, VP-SDE).

# Outline

01 Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)

**02 Probability Flow ODE**

03 Reverse SDE

04 Score-Based Generative Models Through SDEs

05 Denoiser Parameterization and Sampling

06 Flow Matching (FM)

# Probability Flow ODE

## ODE and Continuity Equation

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt$$

$$\frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) \Leftrightarrow \frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}(\mathbf{v}(\mathbf{x}, t)p_t(\mathbf{x}))$$

The only source of randomness is the initial distribution $p_0(\mathbf{x})$.

## SDE and KFP Equation

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}(\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})) + \frac{1}{2}g^2(t)\Delta_{\mathbf{x}} p_t(\mathbf{x})$$

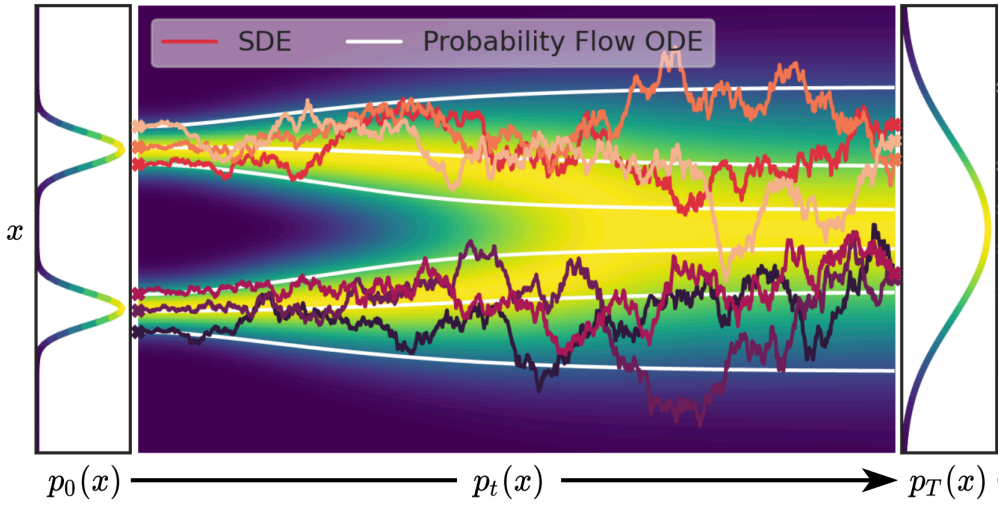
Now there are two sources of randomness: the initial distribution $p_0(\mathbf{x})$ and the Wiener process $\mathbf{w}(t)$.

# Probability Flow ODE

## Theorem

Suppose the SDE $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$ induces the probability path $p_t(\mathbf{x})$ with $p_t(\mathbf{x}) > 0$. Then, there exists an ODE with the same probability path $p_t(\mathbf{x})$, given by

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt = \left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)dt$$



# Probability Flow ODE

## Theorem

Suppose the SDE $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$ induces the probability path $p_t(\mathbf{x})$ with $p_t(\mathbf{x}) > 0$. Then, there exists an ODE with the same probability path $p_t(\mathbf{x})$, given by

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt = \left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)dt$$

## Proof

$$\begin{aligned}
\frac{\partial p_t(\mathbf{x})}{\partial t} &= \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right] + \frac{1}{2}g^2(t)\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right) \\
&= \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x}) - \frac{1}{2}g^2(t)\frac{\partial p_t(\mathbf{x})}{\partial \mathbf{x}}\right]\right) \\
&= \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x}) - \frac{1}{2}g^2(t)p_t(\mathbf{x})\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right]\right) \\
&= \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)p_t(\mathbf{x})\right]\right)
\end{aligned}$$

# Probability Flow ODE

## Proof (Continued)

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)p_t(\mathbf{x})\right]\right)$$

$$= \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{v}(\mathbf{x}, t)p_t(\mathbf{x})\right]\right) = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, t)p_t(\mathbf{x})\right)$$

$$\mathbf{v}(\mathbf{x}, t) = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x}); \quad \tilde{g}(t) = 0$$

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt + 0 \cdot d\mathbf{w} = \left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)dt$$

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}(\mathbf{v}(\mathbf{x}, t)p_t(\mathbf{x})) + \frac{1}{2}\tilde{g}^2(t)\Delta_{\mathbf{x}}p_t(\mathbf{x})$$

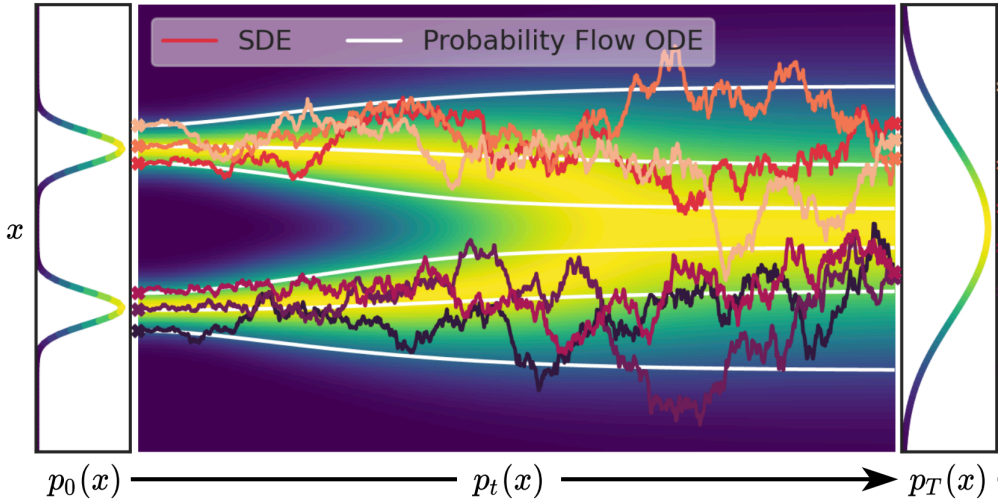
# Probability Flow ODE

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w} \quad - \text{SDE}$$

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt \quad - \text{Probability Flow ODE}$$

$$\mathbf{v}(\mathbf{x}, t) = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})$$

- $\mathbf{s}(\mathbf{x}, t) = \frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})$ is the continuous-time score function.
- The ODE produces more stable trajectories.



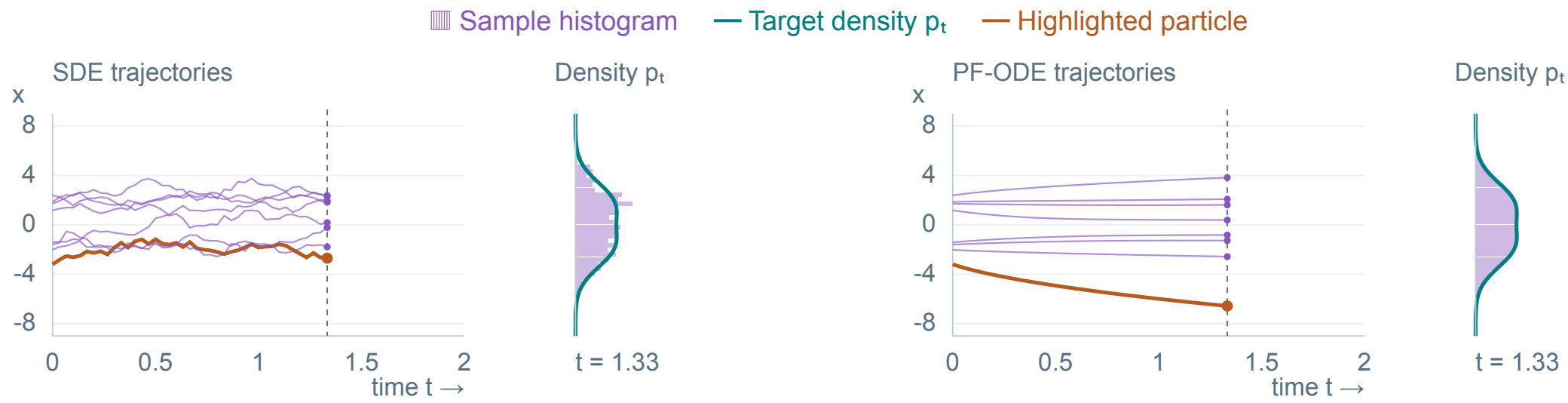

Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

# Same Densities, Different Trajectories

The probability flow ODE follows the same probability path as the SDE.

$$d\mathbf{x} = \sqrt{2}d\mathbf{w}, \qquad d\mathbf{x} = -\mathbf{s}(\mathbf{x}, t)dt$$

Same initial samples; shown from t = 0 to t = 1.33.




The same density at every time does not require the same particle trajectories.

Two-Gaussian data; exact Brownian increments and PF-ODE quantiles. 384 particles; sampling fluctuations remain.

# Why convert SDEs to PF-ODEs? (1) Efficient Sampling

Use the familiar ODE solvers with the learned PF-ODE field:

$$\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t) = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g^2(t) \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, t).$$

| Solver | How it uses the network |
|---|---|
| **Euler / Heun** | Apply the familiar ODE updates: 1 / 2 evaluations per step. EDM uses Heun. |
| **DPM-Solver**<br>**DPM-Solver++** | For VE/VP, integrate the linear drift exactly; approximate the network term. The ++ variant targets guided sampling. |
| **UniPC** | Correct the predicted ODE step using stored network predictions. |

Compare **NFE** (network evaluations). DPM-Solver reports **10–20 NFE**; the required budget depends on the model.

Noise injection is a separate choice: it can improve quality, but does not help every model.

Karras T., et al. Elucidating the Design Space of Diffusion-Based Generative Models, 2022
Lu C., et al. DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps, 2022
Lu C., et al. DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models, 2022
Zhao W., et al. UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models, 2023

# Why convert SDEs to PF-ODEs? (2) Deterministic Sampling

- The PF-ODE defines a deterministic, invertible map $\mathbf{x}(0) \leftrightarrow \mathbf{x}(T)$.
- Each terminal $\mathbf{x}(T)$ produces a unique trajectory back to data space.

## Log-likelihood via the probability-flow ODE

$$\log p_0(\mathbf{x}(0)) = \log p_T(\mathbf{x}(T)) + \int_0^T \operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt$$

Exact change-of-variables identity; numerical evaluation uses an ODE solver and, optionally, Hutchinson's trace estimator.

- Connects PF-ODE to continuous normalizing flows.
- Invertibility enables image $\rightarrow$ latent $\rightarrow$ edited image workflows.
- Smooth interpolation in latent space between two samples.

Chen R. T. Q., et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

# Sampling: Reverse SDE vs PF-ODE

The same trained score $\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, t)$ defines two sampling dynamics (reverse SDE derived next).

## Sampling (Reverse SDE, Euler–Maruyama)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Initialize step $\Delta t = T/K$.
3. Denoise:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Big[\mathbf{f}(\mathbf{x}_t, t) - g^2(t)\, \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\Big]\Delta t + g(t)\sqrt{\Delta t}\cdot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$

## Sampling (PF-ODE, Euler)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Initialize step $\Delta t = T/K$.
3. Denoise:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Big[\mathbf{f}(\mathbf{x}_t, t) - \tfrac{1}{2}\, g^2(t)\, \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\Big]\Delta t \quad \text{(no noise injection)}$$

# Outline

01 Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)

02 Probability Flow ODE

**03 Reverse SDE**

04 Score-Based Generative Models Through SDEs

05 Denoiser Parameterization and Sampling

06 Flow Matching (FM)

# Reverse SDE

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt, \qquad \mathbf{x}(t + dt) = \mathbf{x}(t) + \mathbf{v}(\mathbf{x}, t)dt$$

Here $dt$ can be $> 0$ or $< 0$.

## Reverse ODE

Let $\tau = 1 - t$ ($d\tau = -dt$).

$$d\mathbf{x} = -\mathbf{v}(\mathbf{x}, 1 - \tau)d\tau$$

- How do we reverse the SDE $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$?
- The Wiener process introduces randomness that must be reversed.

## Theorem

There exists a reverse SDE for $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$ (assume $p_t(\mathbf{x}) > 0$), given by:

$$d\mathbf{x} = \left(\mathbf{f}(\mathbf{x}, t) - g^2(t)\frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})\right) dt + g(t)d\bar{\mathbf{w}}$$

where $dt < 0$ and $\bar{\mathbf{w}}$ is a reverse-time Wiener process.

Anderson B. D. O. Reverse-time diffusion equation models, 1982
Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

# Reverse SDE

## Theorem

There exists a reverse SDE for $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$ (assume $p_t(\mathbf{x}) > 0$), given by:

$$d\mathbf{x} = \left(\mathbf{f}(\mathbf{x}, t) - g^2(t)\frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})\right) dt + g(t)d\bar{\mathbf{w}}$$

where $dt < 0$ and $\bar{\mathbf{w}}(t)$ is a reverse-time Wiener process.

- Here the score function appears: $\mathbf{s}(\mathbf{x}, t) = \frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})$.
- Since $\mathbf{v}(\mathbf{x}, t) = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t)$ (PF-ODE), the reverse drift equals $\mathbf{f} - g^2\mathbf{s} = \mathbf{v} - \frac{1}{2}g^2\mathbf{s}$.

## Proof Sketch

- Convert the initial SDE to a probability flow ODE.
- Reverse the probability flow ODE.
- Convert the reversed probability flow ODE back to an SDE.

Anderson B. D. O. Reverse-time diffusion equation models, 1982
Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

# Proof of Reverse SDE

- Convert the initial SDE to a probability flow ODE:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

$$d\mathbf{x} = \left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)dt$$

- Reverse the probability flow ODE:

$$d\mathbf{x} = \left(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\right)dt$$

$$d\mathbf{x} = \left(-\mathbf{f}(\mathbf{x}, 1-\tau) + \frac{1}{2}g^2(1-\tau)\frac{\partial}{\partial \mathbf{x}}\log p_{1-\tau}(\mathbf{x})\right)d\tau$$

# Proof of Reverse SDE

- Convert the reversed probability flow ODE back to an SDE:

$$d\mathbf{x} = \left( -\mathbf{f}(\mathbf{x}, 1-\tau) + \frac{1}{2} g^2(1-\tau) \frac{\partial}{\partial \mathbf{x}} \log p_{1-\tau}(\mathbf{x}) \right) d\tau$$

$$d\mathbf{x} = \left( -\mathbf{f}(\mathbf{x}, 1-\tau) + g^2(1-\tau) \frac{\partial}{\partial \mathbf{x}} \log p_{1-\tau}(\mathbf{x}) \right) d\tau + g(1-\tau) d\mathbf{w}$$

Here $\mathbf{w}(\tau)$ is a Wiener process in forward $\tau$-time.

# Reverse SDE

## Theorem

There exists a reverse SDE for $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$ (assume $p_t(\mathbf{x}) > 0$), given by:

$$d\mathbf{x} = \left(\mathbf{f}(\mathbf{x}, t) - g^2(t)\frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})\right) dt + g(t)d\bar{\mathbf{w}}$$

where $dt < 0$ and $\bar{\mathbf{w}}(t)$ is a reverse-time Wiener process.

## Proof (Continued)

$$d\mathbf{x} = \left(-\mathbf{f}(\mathbf{x}, 1-\tau) + g^2(1-\tau)\frac{\partial}{\partial \mathbf{x}} \log p_{1-\tau}(\mathbf{x})\right) d\tau + g(1-\tau)d\mathbf{w}$$

$$d\mathbf{x} = \left(\mathbf{f}(\mathbf{x}, t) - g^2(t)\frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})\right) dt + g(t)d\bar{\mathbf{w}}$$

Here $d\tau > 0$, $dt < 0$, and $\bar{\mathbf{w}}$ is reverse-time Wiener process.

Anderson B. D. O. Reverse-time diffusion equation models, 1982
Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

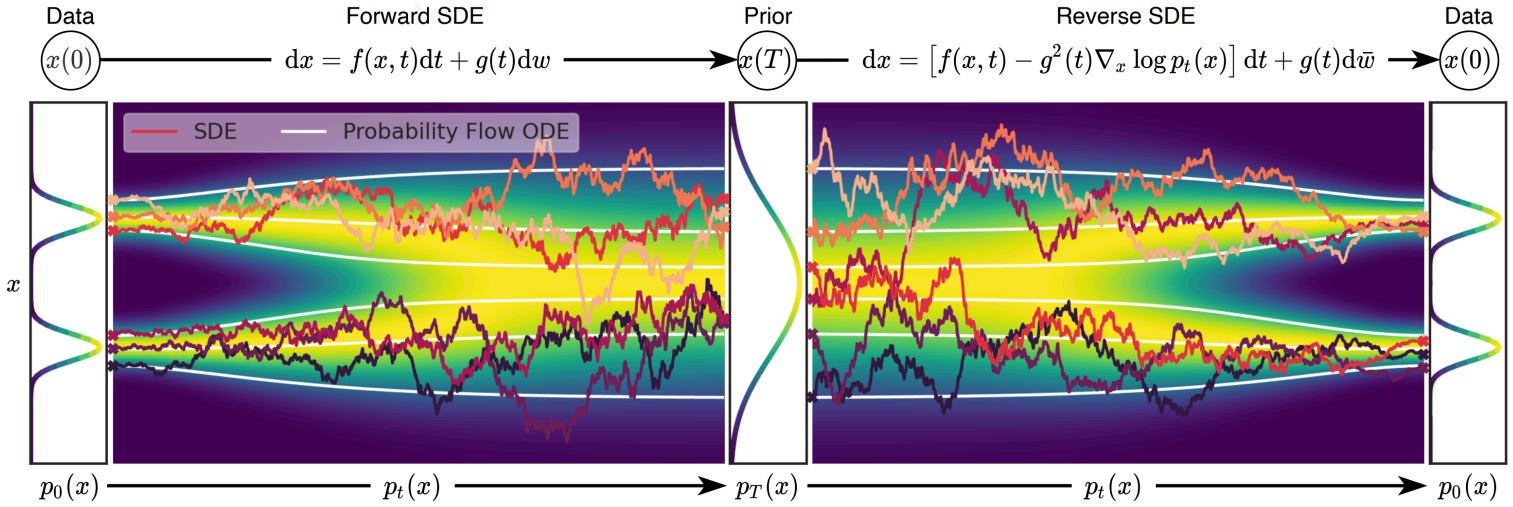
# Reverse SDE

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

$$\mathbf{v}(\mathbf{x}, t) = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t), \quad \text{where} \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

$$d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt \quad \textbf{– Probability Flow ODE}$$

$$d\mathbf{x} = \Big(\mathbf{v}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t)\Big)dt + g(t)d\bar{\mathbf{w}} \quad \textbf{– Reverse SDE}$$

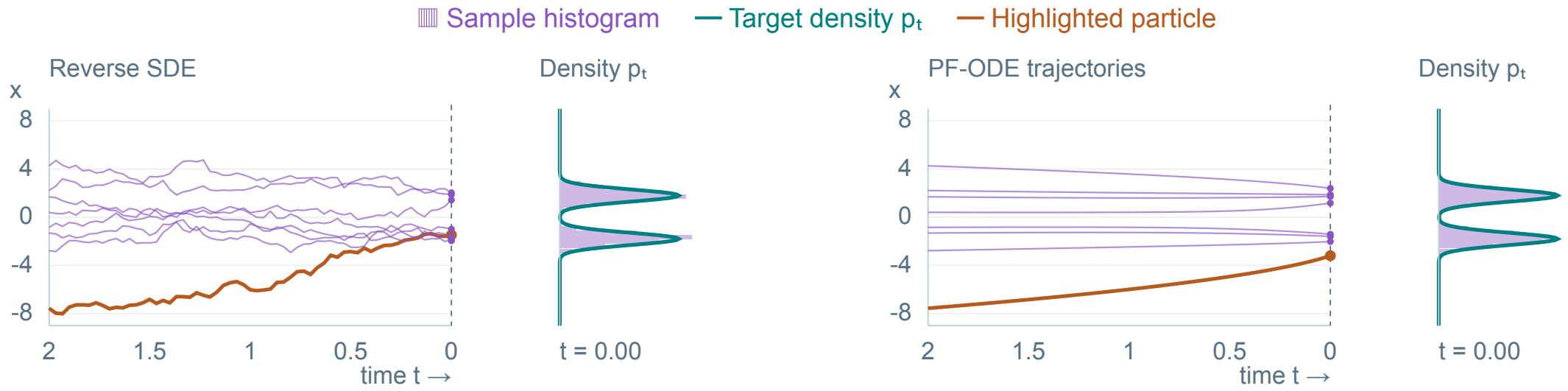


# Reverse SDE and Probability Flow ODE

Start from the same noisy samples: two dynamics, the same evolving density.

$$\text{Reverse SDE:} \quad d\mathbf{x} = 2\mathbf{s}(\mathbf{x}, 2-\tau)d\tau + \sqrt{2}d\mathbf{w}, \quad t = 2-\tau$$

From the same terminal samples at t = 2 back to t = 0.




Reverse SDE and PF-ODE follow the same densities back to data, with different particle trajectories.

Exact reverse conditional transitions; PF-ODE via quantiles. Reverse SDE does not retrace individual forward paths.

# VE and VP: Probability Flow and Reverse SDE

Write $\mathbf{s} = \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$.

## VE / NCSN

$\sigma(t)$ is monotonically increasing.

$$d\mathbf{x} = \sqrt{\frac{d\sigma^2(t)}{dt}}\, d\mathbf{w}$$

## Probability Flow ODE

$$d\mathbf{x} = -\frac{1}{2}\frac{d\sigma^2(t)}{dt}\mathbf{s}\, dt$$

## Reverse SDE

$$d\mathbf{x} = -\frac{d\sigma^2(t)}{dt}\mathbf{s}\, dt + \sqrt{\frac{d\sigma^2(t)}{dt}}\, d\bar{\mathbf{w}}$$

Here $dt < 0$ and $\bar{\mathbf{w}}$ is a reverse-time Wiener process.

## VP / DDPM

Unit initial variance is preserved.

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}\, dt + \sqrt{\beta(t)}\, d\mathbf{w}$$

$$d\mathbf{x} = \left(-\frac{1}{2}\beta(t)\mathbf{x} - \frac{1}{2}\beta(t)\mathbf{s}\right) dt$$

$$d\mathbf{x} = \left(-\frac{1}{2}\beta(t)\mathbf{x} - \beta(t)\mathbf{s}\right) dt + \sqrt{\beta(t)}\, d\bar{\mathbf{w}}$$

# Outline

01 Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)

02 Probability Flow ODE

03 Reverse SDE

**04 Score-Based Generative Models Through SDEs**

05 Denoiser Parameterization and Sampling

06 Flow Matching (FM)

# Score-Based Generative Models Through SDEs

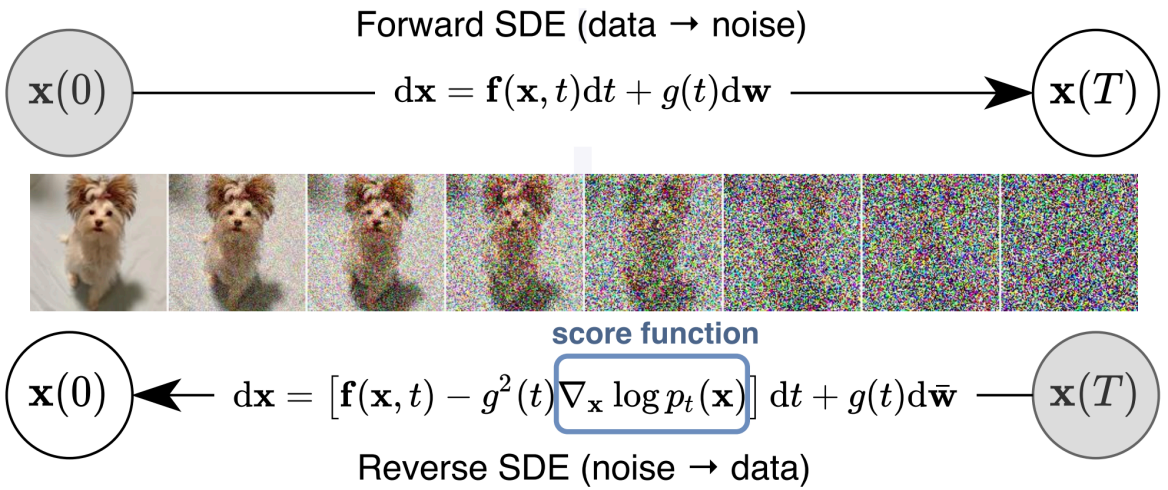
## Discrete-Time Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)}\mathbb{E}_{t\sim U\{1,T\}}\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left\|\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0)\right\|_2^2$$

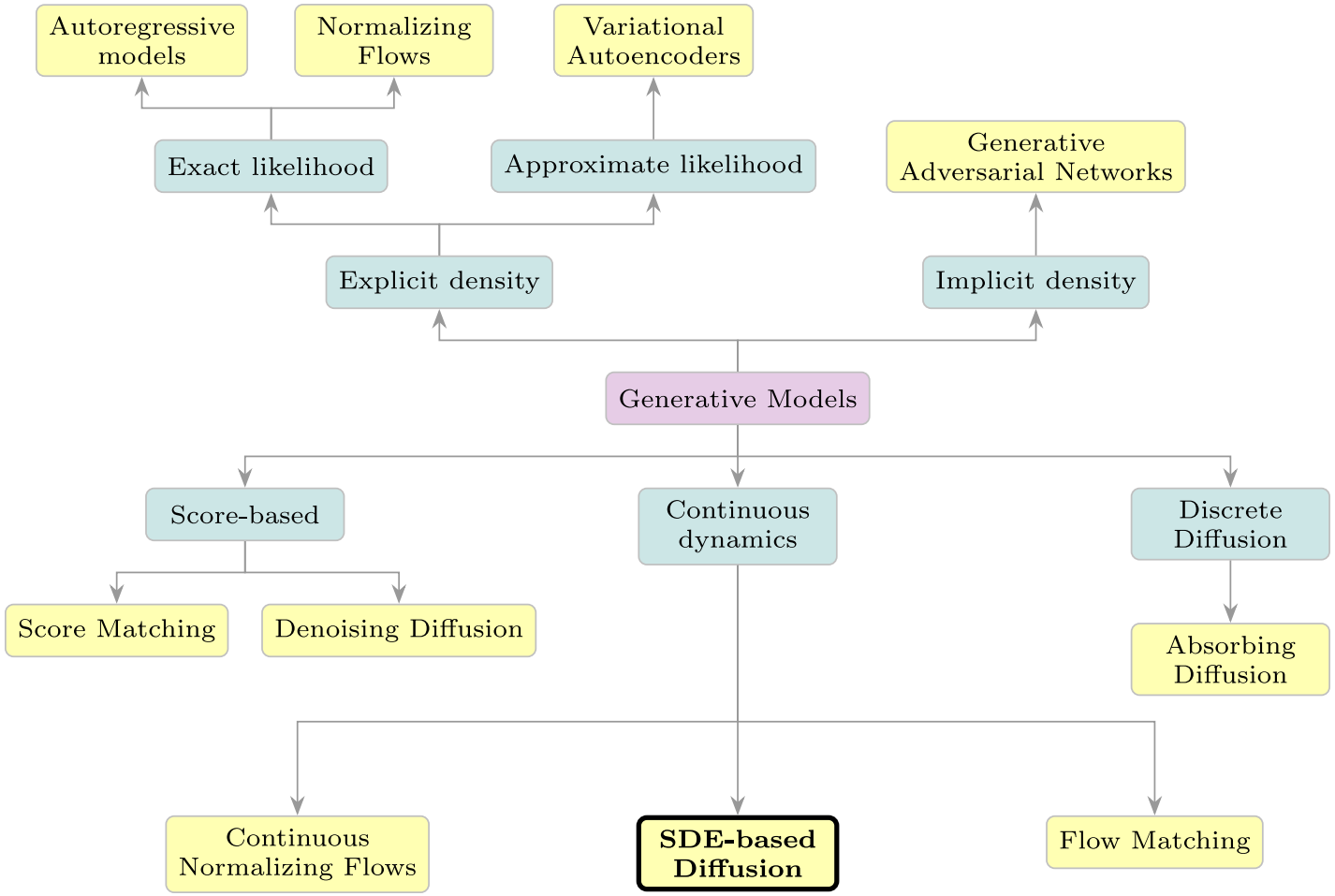
Is it possible to train score-based diffusion models in continuous time?

## Continuous-Time Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t) - \nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$



# Generative Models Taxonomy



# Score-Based Generative Models Through SDEs

## Continuous-Time Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t) - \nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\Big(\boldsymbol{\mu}(\mathbf{x}(0),t), \sigma^2(t)\cdot\mathbf{I}\Big)$$

$$\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0)) = -\frac{1}{\sigma^2(t)}(\mathbf{x}(t)-\boldsymbol{\mu})$$

**Note:** Normality holds for $\mathbf{f}(\mathbf{x},t)$ affine in $\mathbf{x}$ ($\mathbf{f}(\mathbf{x},t) = f(t)\mathbf{x}$).

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}}\cdot d\mathbf{w} \;\; \text{(Variance Exploding SDE)}$$

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)}\cdot d\mathbf{w} \;\; \text{(Variance Preserving SDE)}$$

Is it possible to explicitly derive $\boldsymbol{\mu}(\mathbf{x}(0),t)$ and $\sigma^2(t)$ for VE-SDE and VP-SDE?

# Score-Based Generative Models Through SDEs

$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\Big(\boldsymbol{\mu}(\mathbf{x}(0), t), \sigma^2(t) \cdot \mathbf{I}\Big)$$

## Theorem

The moments of the SDE $d\mathbf{x} = f(t)\mathbf{x}dt + g(t)d\mathbf{w}$ satisfy:

$$\frac{d\boldsymbol{\mu}(\mathbf{x}(0), t)}{dt} = f(t)\boldsymbol{\mu}(\mathbf{x}(0), t), \qquad \frac{d\sigma^2(t)}{dt} = 2f(t)\sigma^2(t) + g^2(t)$$

## Proof

$$\begin{aligned}
\mathbb{E}\left[d\mathbf{x}|\mathbf{x}(0)\right] &= \mathbb{E}\left[f(t)\mathbf{x}dt|\mathbf{x}(0)\right] + \mathbb{E}\left[g(t)d\mathbf{w}|\mathbf{x}(0)\right] \\
&= f(t)\mathbb{E}\left[\mathbf{x}|\mathbf{x}(0)\right]dt + g(t)\mathbb{E}\left[d\mathbf{w}|\mathbf{x}(0)\right] \\
&= f(t)\boldsymbol{\mu}(\mathbf{x}(0), t)dt \\
\frac{d\mathbb{E}\left[\mathbf{x}(t)|\mathbf{x}(0)\right]}{dt} &= \frac{d\boldsymbol{\mu}(\mathbf{x}(0), t)}{dt} = f(t)\boldsymbol{\mu}(\mathbf{x}(0), t)
\end{aligned}$$

We prove the mean equation only; the variance equation is stated without proof.

# Score-Based Generative Models Through SDEs

## Theorem

$$\frac{d\boldsymbol{\mu}(\mathbf{x}(0), t)}{dt} = f(t)\boldsymbol{\mu}(\mathbf{x}(0), t)$$

## Examples

$$\textbf{NCSN:} \quad f(t) = 0 \quad \Rightarrow \quad \boldsymbol{\mu} = \mathbf{x}(0)$$

$$\textbf{DDPM:} \quad f(t) = -\frac{1}{2}\beta(t) \quad \Rightarrow \quad \frac{d\boldsymbol{\mu}}{dt} = -\frac{1}{2}\beta(t)\boldsymbol{\mu}$$

$$\boldsymbol{\mu} = \mathbf{x}(0)\exp\left(-\frac{1}{2}\int_0^t \beta(s)ds\right)$$

# Score-Based Generative Models Through SDEs

## Training

1. Sample $\mathbf{x}(0) \sim p_{\text{data}}(\mathbf{x})$, $t \sim U[0, 1]$.
2. Sample $\mathbf{x}(t) \sim q(\mathbf{x}(t)|\mathbf{x}(0))$.
3. Compute loss $\mathcal{L} = \left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$.

$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\Big(\boldsymbol{\mu}(\mathbf{x}(0), t), \sigma^2(t) \cdot \mathbf{I}\Big)$$

## NCSN

$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\left(\mathbf{x}(0), \left[\sigma^2(t) - \sigma^2(0)\right] \cdot \mathbf{I}\right)$$

## DDPM

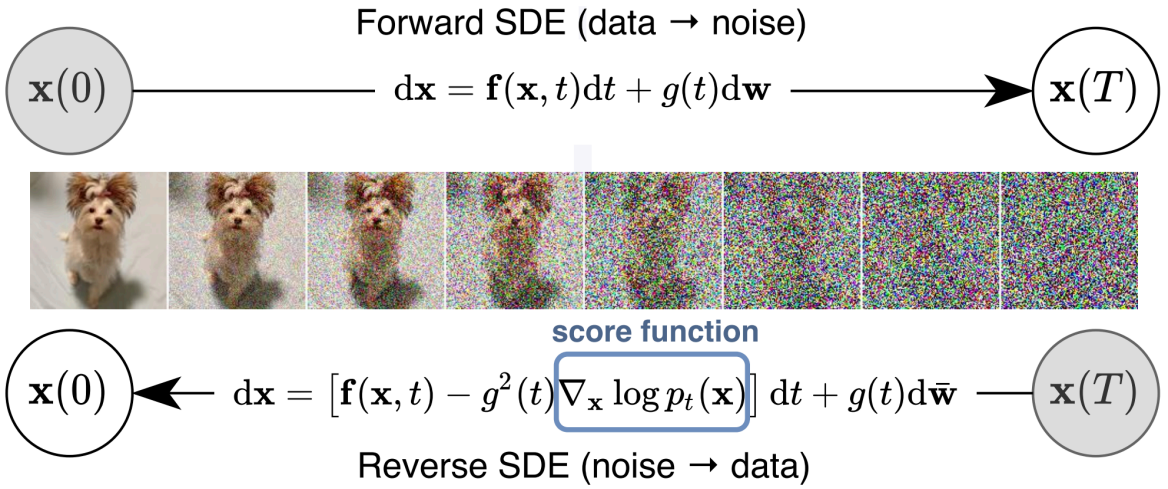
$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\left(\mathbf{x}(0) e^{-\frac{1}{2}\int_0^t \beta(s) ds}, \left(1 - e^{-\int_0^t \beta(s) ds}\right) \cdot \mathbf{I}\right)$$

# Sampling with the Reverse SDE

**Use the trained score for sampling.** Assume terminal prior $\mathcal{N}(0, \mathbf{I})$.

## Sampling

1. Sample $\mathbf{x}(1) \sim \mathcal{N}(0, \mathbf{I})$.
2. Solve the reverse SDE using numerical solvers (`SDESolve`).




- Discretizing the reverse SDE provides ancestral sampling.
- Discretizing the probability flow ODE yields deterministic sampling.

Song Y., et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

# Outline

01 Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)

02 Probability Flow ODE

03 Reverse SDE

04 Score-Based Generative Models Through SDEs

**05 Denoiser Parameterization and Sampling**

06 Flow Matching (FM)

# From Time to Noise Level

**Use the familiar additive-noise form: $\mathbf{x} = \mathbf{x}_0 + \sigma\boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.**

$$\mathbf{f}(\mathbf{x}, t) = 0, \qquad g^2(t) = \frac{d\sigma^2(t)}{dt} = 2\sigma(t)\frac{d\sigma(t)}{dt}.$$

**Rewrite the probability flow ODE using $\sigma$**

$$\begin{aligned}\frac{d\mathbf{x}(t)}{dt} &= \mathbf{f}(\mathbf{x}(t), t) - \frac{1}{2}g^2(t)\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), \sigma(t)) \\ &= -\sigma(t)\frac{d\sigma(t)}{dt}\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), \sigma(t)).\end{aligned}$$

$$\frac{d\mathbf{x}}{d\sigma} = \left(\frac{d\sigma(t)}{dt}\right)^{-1}\frac{d\mathbf{x}(t)}{dt} = -\sigma\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, \sigma).$$

**Choose a denoiser using Tweedie's formula**

$$\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}] = \mathbf{x} + \sigma^2\nabla_{\mathbf{x}}\log p_\sigma(\mathbf{x}).$$

$$D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) \approx \mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}], \qquad \frac{d\mathbf{x}}{d\sigma} = -\sigma\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) = \frac{\mathbf{x} - D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma)}{\sigma}.$$

Decreasing $\sigma$ moves towards the denoiser prediction. **How do we train this denoiser?**

# Train the Denoiser

**Use the existing score-training objective.** For a noisy-clean pair $(\mathbf{x}, \mathbf{x}_0)$ with $q(\mathbf{x} \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, \sigma^2 \mathbf{I})$:

$$\mathcal{L} = \left\| \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) - \nabla_{\mathbf{x}} \log q(\mathbf{x} \mid \mathbf{x}_0) \right\|_2^2 .$$

**Compute the conditional Gaussian target**

$$\nabla_{\mathbf{x}} \log q(\mathbf{x} \mid \mathbf{x}_0) = -\frac{\mathbf{x} - \mathbf{x}_0}{\sigma^2} .$$

**Substitute the denoiser representation**

$$\begin{aligned} \mathcal{L} &= \left\| \frac{D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) - \mathbf{x}}{\sigma^2} + \frac{\mathbf{x} - \mathbf{x}_0}{\sigma^2} \right\|_2^2 \\ &= \frac{1}{\sigma^4} \left\| D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) - \mathbf{x}_0 \right\|_2^2 . \end{aligned}$$

**Same training objective, different prediction target.**
The trained denoiser now supplies the field for sampling.

# What Does the Optimal Denoiser Predict?

Use uniform empirical data $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$. For fixed $\mathbf{x}$ and $\sigma > 0$, let $a_i = \mathcal{N}(\mathbf{x}; \mathbf{x}_i, \sigma^2 \mathbf{I})$ be the likelihood of the noisy observation.

## Minimize the conditional MSE at $\mathbf{x}$

$$D^*(\mathbf{x}, \sigma) = \arg\min_{\mathbf{y}} \sum_{i=1}^{N} a_i \|\mathbf{y} - \mathbf{x}_i\|_2^2 \quad \Longrightarrow \quad 2 \sum_{i=1}^{N} a_i (\mathbf{y} - \mathbf{x}_i) = 0.$$

$$D^*(\mathbf{x}, \sigma) = \frac{\sum_{i=1}^{N} \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_i\|_2^2}{2\sigma^2}\right) \mathbf{x}_i}{\sum_{j=1}^{N} \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_j\|_2^2}{2\sigma^2}\right)} = \mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}].$$

Set the gradient to zero: the optimum is a **posterior-weighted average**.

## Why train a network?

- The exact formula scans **all $N$ training examples at every sampling step**.
- It describes the empirical distribution; exact sampling only reproduces training examples.

# Generate Samples with a Denoiser

**Use the trained denoiser in the ODE solver.** Start from strong noise and decrease $\sigma$.

## Sampling (Euler)

1. Choose $\sigma_0 > \sigma_1 > \cdots > \sigma_N = 0$ and sample $\mathbf{x}^{(0)} \sim \mathcal{N}(0, \sigma_0^2 \mathbf{I})$.
2. For $i = 0, \ldots, N-1$, evaluate $D_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}, \sigma_i)$ and update:

$$\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} + (\sigma_{i+1} - \sigma_i) \frac{\mathbf{x}^{(i)} - D_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}, \sigma_i)}{\sigma_i}.$$

3. Return $\mathbf{x}^{(N)}$.

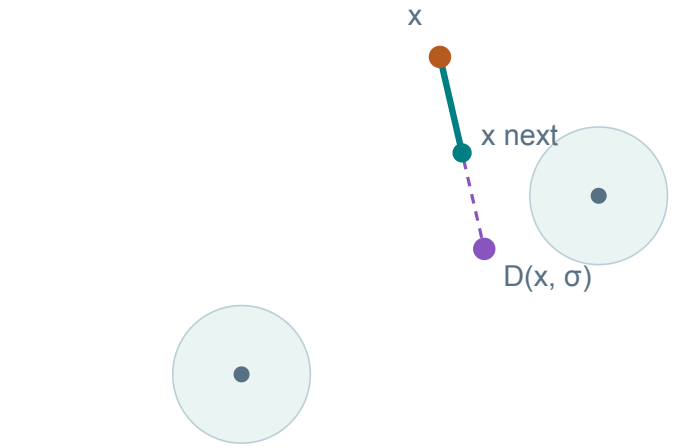
**We now have an explicit Euler sampler in noise coordinates.**
How does this relate to the deterministic DDIM update?

# One Denoiser Prediction, One Euler Step

Decrease the noise level, then recompute the denoiser at the new state.

$$\mathbf{x}_{\text{next}} = r\mathbf{x} + (1-r)D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma), \qquad r = \sigma_{\text{next}}/\sigma$$

One Euler step: σ = 2.5 → 1.25. The next state lies halfway to the current denoiser prediction.




Shaded: clean-data modes

Noise level

**2.50 → 1.25**

Move partway towards the current prediction.

After each step, recompute the denoiser at the new point and noise level.

For r = 0, the final step returns the current prediction.

An Euler sampling step interpolates towards the denoiser prediction. DDIM will give the same update in noise coordinates.

Exact conditional mean for a smooth two-Gaussian toy. The conditional mean can lie between modes.

# DDIM Is Euler in Noise Coordinates

## Recall deterministic DDIM ($s < t$):

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \qquad \hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}},$$

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s}\,\hat{\boldsymbol{\epsilon}}.$$

## Recall the DDPM / NCSN rescaling ($\sigma_s < \sigma_t$)

$$\tilde{\mathbf{x}}_u = \frac{\mathbf{x}_u}{\sqrt{\bar{\alpha}_u}}, \qquad \sigma_u = \sqrt{\frac{1-\bar{\alpha}_u}{\bar{\alpha}_u}}, \qquad u \in \{s, t\}.$$

$$D_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_t, \sigma_t) = \hat{\mathbf{x}}_0 = \tilde{\mathbf{x}}_t - \sigma_t \hat{\boldsymbol{\epsilon}}.$$

## Rewrite DDIM as the denoiser Euler step

$$\begin{aligned}\tilde{\mathbf{x}}_s &= \hat{\mathbf{x}}_0 + \sigma_s \hat{\boldsymbol{\epsilon}} = \tilde{\mathbf{x}}_t + (\sigma_s - \sigma_t)\hat{\boldsymbol{\epsilon}} \\ &= \tilde{\mathbf{x}}_t + (\sigma_s - \sigma_t)\frac{\tilde{\mathbf{x}}_t - D_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_t, \sigma_t)}{\sigma_t}.\end{aligned}$$

## DDIM implements this Euler step using the existing noise predictor.

# Outline

01 Diffusion and Score Matching SDEs (VP-SDE / VE-SDE)

02 Probability Flow ODE

03 Reverse SDE

04 Score-Based Generative Models Through SDEs

05 Denoiser Parameterization and Sampling

**06 Flow Matching (FM)**

# Continuous-Time Normalizing Flows

**Now learn the velocity field itself.** Recall $\mathbf{x}_t = \mathbf{x}(t)$ for $t \in [0, 1]$:

- $\mathbf{x}_0 \sim p_0(\mathbf{x})$, where $p_0(\mathbf{x})$ is a base distribution (mostly $\mathcal{N}(0, \mathbf{I})$);
- $\mathbf{x}_1 \sim p_1(\mathbf{x})$, where $p_1(\mathbf{x})$ is the true data distribution $p_{\text{data}}(\mathbf{x})$.

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, t), \quad \text{with initial condition } \mathbf{x}(0) = \mathbf{x}_0.$$

The velocity $\mathbf{v}$ determines a flow $\boldsymbol{\psi}_t$: $\mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0)$.

**KFP Theorem (Continuity Equation)**

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, t) p_t(\mathbf{x})\right) \Leftrightarrow \frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right)$$

- Likelihood-based CNF training requires ODE solves and Jacobian trace evaluations.
- The adjoint method computes gradients through the ODE solution; repeated solves can be costly.

# Continuous-Time Normalizing Flows

## KFP Theorem (Continuity Equation)

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, t) p_t(\mathbf{x})\right) \Leftrightarrow \frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right)$$

- Knowing the vector field $\mathbf{v}(\mathbf{x}, t)$, the KFP (or continuity) equation allows us to compute the density $p_t(\mathbf{x})$.
- Flow matching fits velocities by regression, without model ODE solves or trace evaluations during training.

## Flow Matching

$$\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{\mathbf{x} \sim p_t(\mathbf{x})} \left\| \mathbf{v}(\mathbf{x}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t) \right\|^2 \to \min_{\boldsymbol{\theta}}$$

- Approximate the true vector field $\mathbf{v}(\mathbf{x}, t)$ using $\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)$.
- Use the learned flow $\boldsymbol{\psi}_t$ for deterministic sampling: $\mathbf{x}_1 = \boldsymbol{\psi}_1(\mathbf{x}_0)$.

# Flow Matching: Training and Sampling

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})}\left\|\mathbf{v}(\mathbf{x},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

$$\mathbf{x}_0\sim p_0(\mathbf{x})=p(\mathbf{x}),\quad \mathbf{x}_1\sim p_1(\mathbf{x})=p_{\text{data}}(\mathbf{x})$$

## Training

1. Sample $t\sim U[0,1]$, $\mathbf{x}_t\sim p_t(\mathbf{x})$.
2. Compute loss $\mathcal{L}=\left\|\mathbf{v}(\mathbf{x}_t,t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2$.

## Sampling

1. Sample $\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1=\boldsymbol{\psi}_1(\mathbf{x}_0)=\texttt{ODESolve}_v(\mathbf{x}_0,\boldsymbol{\theta},t_0=0,t_1=1).$$

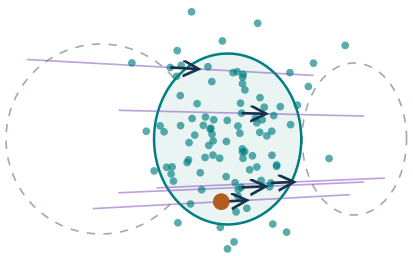
**Problem:** The true vector field $\mathbf{v}(\mathbf{x},t)$ is **unknown**.

# Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})}\left\|\mathbf{v}(\mathbf{x},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

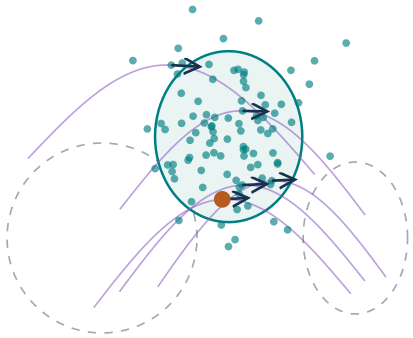
All three flows at t = 0.5. Dashed contours mark the common endpoint densities.

**Direct**



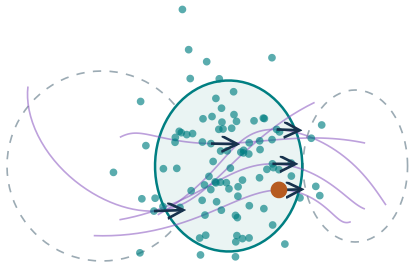
$p_0$ · noise $p_1$ · data

**Detour**



$p_0$ · noise $p_1$ · data

**Rotation**



$p_0$ · noise $p_1$ · data

Many velocity fields connect the same endpoints. Choose a path with a tractable training objective.

Intermediate densities may differ. Purple: trajectories; arrows: velocities; orange: one shared particle.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022
Original illustration: A Visual Dive into Conditional Flow Matching

# Summary

- Score matching (NCSN) and diffusion models (DDPM) are discretizations of SDEs (variance exploding and variance preserving).
- Every SDE admits a corresponding probability flow ODE that follows the same probability path $p_t(\mathbf{x})$, yielding deterministic invertible sampling and log-likelihood evaluation via the instantaneous change of variables.
- A denoiser parameterizes the PF-ODE; DDIM is Euler in rescaled state and noise coordinates. Compare solver cost in network evaluations (NFE).
- SDEs can be reversed in time using the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$; discretizing the reverse SDE gives ancestral sampling, while discretizing the PF-ODE gives deterministic sampling.
- The continuous-time score matching objective generalizes DDPM and NCSN; for affine-drift SDEs the transition kernel $q(\mathbf{x}(t)|\mathbf{x}(0))$ is Gaussian, making the training objective tractable.
- Flow matching trains the velocity field by regression, avoiding model ODE solves and trace evaluations during training.